Herausforderungen

Herausforderungen

Neun Stück, von leicht nach schwer.

Come si usa — per te

Le nove sfide sono in ordine di difficoltà, non di argomento. Si danno **una per volta**, mai tutte insieme: il foglio intero gli dice “ecco quanto ti manca”, una sfida alla volta gli dice “ecco la prossima”.

Le prime due sono **Schrägbilder**. Non è materiale avanzato: è il Lernbereich 3 di matematica di sesta, quello che sta facendo proprio adesso. Parte da lì perché sa già come deve venire il disegno — deve solo scoprire come dirlo alla macchina.

Dalla terza in poi entra il **ciclo**, dalla settima la **doppia condizione** con `if` e `else`. Nessuna sfida usa seno, coseno o radice quadrata: non sono materia di sesta e non servono.

Ogni sfida ha tre parti: cosa costruire, un riquadro con l’aiuto, e una **domanda aperta** in fondo. La domanda è la parte che conta. Quando ti chiama, non risolvergli il problema: mandalo al riquadro, e poi chiedigli la domanda in fondo.

Quando ha finito, non chiedergli di spiegarti cosa ha fatto. Chiedigli di **fartelo vedere**, e di cambiare un numero davanti a te.

Le soluzioni stanno nella seconda metà di questo foglio. Sono per te, non per lui.

HERAUSFORDERUNGEN

1 · Der Würfel

Du kennst das Schrägbild aus Mathe: die Tiefenkanten gehen schräg nach hinten, und sie sind kürzer als in Wirklichkeit. Genau das machen wir jetzt mit der Schildkröte.

Bau einen Würfel: die vordere Fläche, die hintere Fläche, und die vier Kanten dazwischen.

> **Hilfe:** Schräg nach hinten heißt: *gleich viel nach rechts wie nach oben.* Wenn `versatz = 56` ist, dann fängt die hintere Fläche bei `start_x + 56` und `start_y + 56` an. Mehr brauchst du nicht.

Frage: Ändere nur `versatz`. Wann sieht es aus wie ein Würfel? Wann wie ein platt gedrücktes Brett?

---

2 · Das Haus

Dasselbe, aber mit einem Haus: vorne eine Wand mit Dach, hinten dieselbe Wand mit Dach, dazwischen die Tiefenkanten — und ganz oben der Dachfirst.

> **Hilfe:** Du schreibst dieselben vier Zeilen immer wieder: `penup, goto, pendown, setheading`. Gib ihnen einen Namen, dann schreibst du sie nur einmal:
>
> ```
> def nach(x, y):
>     penup()
>     goto(x, y)
>     pendown()
>     setheading(0)
> ```
>
> Ab jetzt reicht `nach(100, 50)`.

Frage: Der Dachfirst ist eine einzige Linie. Wo fängt sie an, wo hört sie auf? Finde die beiden Punkte, bevor du sie zeichnest.

---

3 · Der Turm

Stapel Würfel übereinander. Nicht mit Kopieren und Einfügen — mit einer **Schleife**.

Und: jedes zweite Stockwerk soll eine andere Farbe haben.

> **Hilfe:** `stock % 2` ist der *Rest* beim Teilen durch 2. Der ist 0, 1, 0, 1, 0, 1 …
>
> ```
> if stock % 2 == 0:
>     color("darkblue")
> else:
>     color("firebrick")
> ```
>
> Ein `=` setzt etwas. Zwei `==` fragen nach.

Frage: Jedes Stockwerk steht genau `kante` höher als das darunter. Welche Zahl schreibst du dafür in `goto`?

---

4 · Die Spirale

Eine Schleife, bei der jeder Schritt länger ist als der vorige.

> **Hilfe:** In `for i in range(schritte):` zählt `i` mit: 0, 1, 2, 3, 4 … Schreib `forward(i * 2)` statt einer festen Zahl.

Frage: Nimm `right(90)` — es wird ein eckiges Schneckenhaus. Nimm jetzt `right(89)`. Was passiert, und warum ausgerechnet das?

---

5 · Das Vieleck im Kreis

Zeichne ein Sechseck, das **genau** in einen Kreis passt: jede Ecke berührt die Kreislinie, keine ragt heraus.

> **Hilfe:** `circle()` kann drei Zahlen vertragen, nicht nur eine. Probier `circle(200, 360, 6)` und finde heraus, was die dritte macht.

Frage: Zeichne den Kreis grau darüber. Berühren sich die Ecken wirklich? Und bei 3 Ecken? Bei 36?

---

6 · Der Stern

Ein Fünfeck entsteht, wenn du von Ecke zu Ecke gehst. Ein **Stern** entsteht, wenn du dabei eine Ecke überspringst.

> **Hilfe:** `right(360 * sprung / ecken)`
>
> `sprung = 1` gibt das normale Vieleck. `sprung = 2` überspringt jedes Mal eine Ecke.

Frage: `ecken = 5, sprung = 2` schließt sich zu einem Stern. `ecken = 6, sprung = 2` nicht — da kommt ein Dreieck heraus, das zweimal gezeichnet wird. Probier 7/2, 7/3, 8/2, 8/3, 9/3. Welche schließen sich, welche nicht? **Was haben die kaputten gemeinsam?**

---

7 · Das Schachbrett

Acht mal acht Felder, abwechselnd schwarz und weiß.

> **Hilfe:** Eine Schleife für die Zeilen, und **in** ihr noch eine für die Spalten:
>
> ```
> for zeile in range(8):
>     for spalte in range(8):
>         ...
> ```
>
> Die Farbe entscheidet `(zeile + spalte) % 2`.

Frage: Warum `zeile + spalte` und nicht nur `spalte`? Probier es mit nur `spalte` und schau dir an, was herauskommt.

---

8 · Miss Pi

Ein Kreis ist fast dasselbe wie ein Vieleck mit sehr vielen Ecken. Also: miss den Umfang eines Vielecks und teil ihn durch den Durchmesser.

> **Hilfe:** Du musst nichts ausrechnen — die Schildkröte kann selber messen:
>
> ```
> anfang = pos()
> circle(radius, 360 / ecken, 1)   # genau EINE Seite
> seite = distance(anfang)
> ```

Frage: Trag nacheinander 6, 12, 24, 48, 96 und 360 Ecken ein und schreib die Ergebnisse untereinander auf ein Blatt. Was fällt dir auf? Und: ist deine Zahl immer zu groß oder immer zu klein? **Warum muss das so sein?**

---

9 · Der Pythagoras-Jäger

3, 4, 5 ist ein rechtwinkliges Dreieck mit lauter ganzen Zahlen. Es gibt noch mehr davon. **Finde sie alle bis 30.**

> **Hilfe:** Drei Schleifen ineinander und ein `if`:
>
> ```
> if a * a + b * b == c * c:
>     print(a, b, c)
> ```
>
> Du brauchst keine Wurzel. Probier einfach jedes `c` durch und frag nach.

Frage: Du findest 3-4-5, aber auch 6-8-10 und 9-12-15. Was fällt dir an diesen dreien auf? Welche aus deiner Liste sind wirklich **neu**, und welche sind nur ein vergrößertes Dreieck von vorher?

Le soluzioni

1 · Der Würfel

Il punto non è il cubo, è `versatz`. È l'unico numero che decide come appare la profondità, e lui deve smanettarci: con 10 il cubo è schiacciato, con 150 diventa una scatola lunga.

Nota che non c'è nessun angolo di 45 gradi scritto da nessuna parte. "Tanto a destra quanto in su" è 45 gradi — ed è così che glielo insegnano a matematica.

```
# HERAUSFORDERUNG 1 – Der Würfel im Schrägbild
# versatz: gleich viel nach rechts wie nach oben. Das sind die 45 Grad.
from turtle import *

kante = 150
versatz = 56        # <-- probier 30, 56, 90
start_x = -160
start_y = -140

speed(0)
pensize(3)
color("darkblue")

# die vordere Fläche
penup()
goto(start_x, start_y)
pendown()
setheading(0)
for i in range(4):
    forward(kante)
    left(90)

# die hintere Fläche, verschoben
penup()
goto(start_x + versatz, start_y + versatz)
pendown()
setheading(0)
for i in range(4):
    forward(kante)
    left(90)

# die vier Tiefenkanten
for ecke in [(start_x, start_y),
             (start_x + kante, start_y),
             (start_x + kante, start_y + kante),
             (start_x, start_y + kante)]:
    penup()
    goto(ecke)
    pendown()
    goto(ecke[0] + versatz, ecke[1] + versatz)

done()
```

---

2 · Das Haus

Qui serve `def`, perché senza le quattro righe di spostamento si ripetono dodici volte. È il primo posto dove una funzione si giustifica da sola, non perché gliel'hai detto tu.

La trappola è il colmo del tetto (`Dachfirst`): è l'unica linea che va dal tetto davanti al tetto dietro, e parte dalla punta — cioè da `start_x + breite / 2`, non da un angolo.

```
# HERAUSFORDERUNG 2 — Das Haus im Schrägbild
# versatz: gleich viel nach rechts wie nach oben. Das ist der 45-Grad-Trick.
from turtle import *

speed(0)
pensize(3)

breite = 160
hoehe = 140
dach = 80
versatz = 56       # <-- probier 30, 56, 90
start_x = -170
start_y = -150

def nach(x, y):
    penup()
    goto(x, y)
    pendown()
    setheading(0)

def tief(x, y):
    """die Tiefenkante: gleich viel nach rechts wie nach oben"""
    nach(x, y)
    goto(x + versatz, y + versatz)

color("firebrick")

# die vordere Wand
nach(start_x, start_y)
forward(breite)
left(90)
forward(hoehe)
left(90)
forward(breite)
left(90)
forward(hoehe)
left(90)

# die hintere Wand, verschoben
nach(start_x + versatz, start_y + versatz)
forward(breite)
left(90)
forward(hoehe)
left(90)
forward(breite)
left(90)
forward(hoehe)
left(90)

# die vier Tiefenkanten
tief(start_x, start_y)
tief(start_x + breite, start_y)
tief(start_x + breite, start_y + hoehe)
tief(start_x, start_y + hoehe)

color("sienna")

# das vordere Dach
nach(start_x, start_y + hoehe)
goto(start_x + breite / 2, start_y + hoehe + dach)
goto(start_x + breite, start_y + hoehe)

# das hintere Dach
nach(start_x + versatz, start_y + hoehe + versatz)
goto(start_x + breite / 2 + versatz, start_y + hoehe + dach + versatz)
goto(start_x + breite + versatz, start_y + hoehe + versatz)

# der Dachfirst
tief(start_x + breite / 2, start_y + hoehe + dach)

done()
```

---

3 · Der Turm

Prima sfida con ciclo e `if` insieme. `stock % 2` dà 0, 1, 0, 1 — il resto della divisione per due, che in sesta hanno fatto.

Se sbaglia, sbaglia quasi sempre nel punto in cui mette `y`: deve essere `start_y + stock * kante`, cioè dipendere dal contatore. Se scrive `start_y` e basta, i quattro cubi finiscono tutti uno sopra l'altro nello stesso posto e sembra che non sia successo niente.

```
# HERAUSFORDERUNG 3 – Der Turm
# Eine Schleife baut Stockwerk auf Stockwerk.
# if / else gibt jedem zweiten eine andere Farbe.
from turtle import *

kante = 100
versatz = 38
stockwerke = 4      # <-- probier 2, 4, 6
start_x = -140
start_y = -200

speed(0)
pensize(3)

for stock in range(stockwerke):

    if stock % 2 == 0:
        color("darkblue")
    else:
        color("firebrick")

    x = start_x
    y = start_y + stock * kante

    # vorne
    penup()
    goto(x, y)
    pendown()
    setheading(0)
    for i in range(4):
        forward(kante)
        left(90)

    # hinten
    penup()
    goto(x + versatz, y + versatz)
    pendown()
    setheading(0)
    for i in range(4):
        forward(kante)
        left(90)

    # die Tiefenkanten
    for ecke in [(x, y), (x + kante, y),
                 (x + kante, y + kante), (x, y + kante)]:
        penup()
        goto(ecke)
        pendown()
        goto(ecke[0] + versatz, ecke[1] + versatz)

done()
```

4 · Die Spirale

Il salto concettuale è che `i` non è decorativo: è un numero che cambia da solo a ogni giro, e si può usare nel calcolo.

La risposta alla domanda: con 90 gradi dopo quattro passi guarda di nuovo nella stessa direzione, quindi la figura resta dritta. Con 89 gradi ogni giro resta indietro di un grado, e quel grado di errore si accumula — ed è proprio l'errore che fa girare la spirale.

```
# HERAUSFORDERUNG 4 – Die Spirale
# Die Schleife zählt mit: i wird bei jedem Schritt größer.
from turtle import *

winkel = 89
schritte = 120

speed(0)
pensize(2)
color("purple")
penup()
goto(0, 0)
pendown()
for i in range(schritte):
    forward(i * 2)
    right(winkel)

done()
```

5 · Das Vieleck im Kreis

Questa è una scoperta, non un calcolo: `circle(radius, 360, ecken)` disegna il poligono inscritto e nessuno lo sa.

La verifica è visiva: disegna il cerchio grigio sopra, e gli angoli devono toccarlo. Con 36 lati il poligono e il cerchio diventano indistinguibili — ed è il ponte verso la sfida 8.

```
# HERAUSFORDERUNG 5 – Das Vieleck im Kreis
# Das Vieleck soll genau IM Kreis sitzen: jede Ecke berührt die Kreislinie.
from turtle import *

radius = 200
ecken = 6          # <-- probier 3, 5, 8, 12

speed(0)
hideturtle()
pensize(2)

# der Kreis (grau)
color("gray")
penup()
goto(radius, 0)
pendown()
setheading(90)
circle(radius)

# das Vieleck (rot) – die DRITTE Zahl sagt, wie viele Ecken
color("firebrick")
penup()
goto(radius, 0)
pendown()
setheading(90)
circle(radius, 360, ecken)

done()
```

---

6 · Der Stern

La risposta alla domanda è il massimo comun divisore. La figura si chiude passando per tutte le punte solo se `ecken` e `sprung` non hanno divisori in comune.

5/2 funziona (1 e 5 non hanno divisori comuni), 6/2 no (hanno il 2 in comune, e viene un triangolo ripassato due volte), 7/2 e 7/3 funzionano entrambi perché 7 è primo, 8/2 no, 8/3 sì, 9/3 no.

Se ci arriva da solo guardando i numeri, ha scoperto una cosa che in Germania si fa alle superiori.

```
# HERAUSFORDERUNG 6 – Der Stern
# Nicht zur nächsten Ecke, sondern über welche hinweg.
from turtle import *

ecken = 7
sprung = 3

penup()
goto(-100, -120)
pendown()
setheading(0)
pensize(3)
color("darkred")
for i in range(ecken):
    forward(220)
    right(360 * sprung / ecken)

done()
```

---

7 · Das Schachbrett

Primo ciclo dentro un ciclo. Il ciclo interno gira otto volte per ogni singolo giro di quello esterno: sessantaquattro caselle in tutto.

La risposta alla domanda: con solo `spalte` vengono colonne verticali, non una scacchiera, perché ogni riga ricomincia dallo stesso colore. Sommando `zeile` ogni riga parte sfasata di uno. Fateglielo provare, non ditelo.

```
        # HERAUSFORDERUNG 7 — Das Schachbrett
        # Eine Schleife IN einer Schleife. if / else entscheidet die Farbe.
        from turtle import *

        felder = 8
        feld = 50
        start_x = -200
        start_y = -200

        speed(0)
        hideturtle()
        pensize(1)
        color("black")

        for zeile in range(felder):
            for spalte in range(felder):

                if (zeile + spalte) % 2 == 0:
                    fillcolor("white")
                else:
                    fillcolor("black")

                penup()
                goto(start_x + spalte * feld, start_y + zeile * feld)
                pendown()
                setheading(0)
                begin_fill()
                for i in range(4):
                    forward(feld)
                    left(90)
                end_fill()

        done()
```

---

8 · Miss Pi

È il metodo di Archimede, ma senza trigonometria: la tartaruga disegna un lato e poi `distance()` lo misura. La macchina fa da metro.

I numeri che deve ottenere:

Ecken	Umfang / Durchmesser
6	3,000000
12	3,105829
24	3,132629
48	3,139350
96	3,141032
360	3,141553

La risposta alla domanda: il valore è **sempre troppo piccolo**, e non può essere altrimenti. Il poligono sta dentro il cerchio, quindi il suo perimetro è per forza più corto della circonferenza. Più lati metti, più ti avvicini — ma sempre da sotto.

Con 96 lati arriva a 3,141 — che è esattamente dove era arrivato Archimede, a mano, 2200 anni fa.

```
# HERAUSFORDERUNG 8 – Miss Pi
# Python zeichnet EINE Seite und misst sie selbst nach.
from turtle import *

radius = 200
ecken = 6          # <-- probier 6, 12, 24, 48, 96, 360

speed(0)
hideturtle()
pensize(2)
color("firebrick")

penup()
goto(radius, 0)
setheading(90)

anfang = pos()
pendown()
circle(radius, 360 / ecken, 1)    # genau EINE Seite
seite = distance(anfang)

umfang = seite * ecken
durchmesser = 2 * radius

print("Ecken:", ecken)
print("eine Seite:", seite)
print("Umfang:", umfang)
print("Umfang / Durchmesser:", umfang / durchmesser)

# und jetzt das ganze Vieleck dazu
penup()
goto(radius, 0)
pendown()
setheading(90)
circle(radius, 360, ecken)

done()
```

---

9 · Der Pythagoras-Jäger

Tre cicli annidati e un confronto. Niente radice quadrata: invece di calcolare `c`, prova tutti i `c` possibili e chiede se il quadrato torna.

Quello che stampa, fino a 30:

```
3  4  5          12 16 20
5  12 13         15 20 25
6  8  10         16 30 34
7  24 25         18 24 30
8  15 17         20 21 29
9  12 15         21 28 35
10 24 26
```

La risposta alla domanda: 6-8-10 è 3-4-5 moltiplicato per due, 9-12-15 per tre, 12-16-20 per quattro. Sono lo stesso triangolo ingrandito. I veramente nuovi in questa lista sono 3-4-5, 5-12-13, 7-24-25, 8-15-17, 20-21-29 — quelli dove `a`, `b` e `c` non hanno nessun divisore in comune.

La seconda metà del file disegna 9-12-15 e poi lo fa misurare a Python: deve stampare esattamente 300.0. Vedere la macchina confermare il teorema è la parte che gli resta in testa.

E nota: l'ipotenusa non la disegna con un angolo. La chiude con `goto()` tornando al punto di partenza — quindi non gli serve nessuna trigonometria nemmeno per il disegno.

```python
# HERAUSFORDERUNG 9 – Der Pythagoras-Jäger
# Teil 1: alle rechtwinkligen Dreiecke mit ganzen Zahlen suchen.
# Teil 2: eines davon zeichnen und von Python nachmessen lassen.
from turtle import *

bis = 30

print("a    b    c")
for a in range(1, bis + 1):
    for b in range(a, bis + 1):
        for c in range(b, 2 * bis):
            if a * a + b * b == c * c:
                print(a, " ", b, " ", c)

# --- jetzt eines davon zeichnen ---

a = 9
b = 12
faktor = 20

start_x = -100
start_y = -120

speed(0)
pensize(3)
penup()
goto(start_x, start_y)
pendown()
setheading(0)

color("firebrick")
forward(a * faktor)     # die liegende Kathete
left(90)                # der rechte Winkel
forward(b * faktor)     # die stehende Kathete

print("gemessene Hypotenuse:", distance(start_x, start_y))

color("steelblue")
goto(start_x, start_y)  # die Hypotenuse

done()
```

Se divora anche queste

Hai ancora tre direzioni, in ordine di quanto lavoro costano a te.

La più economica: fargli fare l'insegnante. Dagli uno dei file di esempio degli altri con dentro un errore e chiedigli di trovarlo. Oppure, meglio, chiedi a lui di *inventare* la sfida 10 per sé stesso e di scriverne la soluzione. Preparare un problema è più difficile che risolverlo.

La via naturale: l'animazione. Sa già `while`, e gli mancano solo tre cose per far muovere qualcosa sullo schermo: `clear()` per cancellare il disegno vecchio, un ciclo che non finisce, e `onkey()` per la tastiera. Sono esattamente le tre cose che stanno dentro i giochi che hai già in cartella — a quel punto può aprire `pong.py` e capire cosa legge, invece di guardarlo come una scatola nera.

La più lunga: la terza dimensione vera. Finora la profondità è un numero fisso, `versatz`. Un cubo che *gira* è un'altra storia: lì servono davvero seno e coseno, e sono di decima classe. Non è il posto per insegnarglieli. Se ci tiene molto, la versione onesta è dargli le formule come una scatola che funziona, dicendogli chiaramente che il perché arriva fra quattro anni.